High-Performance Linear Algebra Report

Matrix-Vector and Matrix-Matrix Multiplication in C++

Course / Assignment	High-Performance Linear Algebra Kernels
Team Members	Moxiao Li
Repository	https://github.com/MOXIAO1998/Advanced-Computing-for-Finance

This report summarizes the implementation, benchmarking, profiling, and optimization of baseline linear algebra kernels written in C++. The study compares storage layouts, analyzes cache locality and memory alignment, evaluates compiler optimizations, and presents at least one significant optimization to matrix multiplication.

Contents

1 Baseline Implementations **2**

1.1 Matrix-Vector Multiplication . . . 2

1.2 Matrix-Matrix Multiplication . . . 3

1.3 Correctness Tests for the Baseline Implementation . . . 5

2 Benchmarking Methodology **6**

2.1 Experimental Setup . . . 6

2.2 Problem Sizes . . . 6

2.3 Benchmark Tables . . . 7

3 Cache Locality Analysis **10**

3.1 Matrix-Vector Multiplication . . . 10

3.2 Matrix-Matrix Multiplication . . . 10

3.3 Stride-Sensitive Cases . . . 11

4 Memory Alignment **11**

4.1 Method . . . 11

4.2 Discussion . . . 12

5 Inlining and Compiler Optimization **12**

5.1 Compilation Modes . . . 12

5.2 Discussion . . . 13

6 Profiling Results **13**

6.1 Procedure . . . 13

6.2 Findings . . . 15

7 Optimization Strategy and Final Results **16**

7.1 Selected Optimization . . . 16

7.2 Implementation Summary . . . 16

7.3 Interpretation . . . 17

8 Conclusion **18**

1 Baseline Implementations

The baseline code implements the following functions:

- `multiply_mv_row_major`
- `multiply_mv_col_major`
- `multiply_mm_naive`
- `multiply_mm_transposed_b`

All functions use raw pointers for input and output arrays. The test program dynamically allocates and deallocates matrices and vectors, checks correctness on small cases, and includes basic validation such as null-pointer checks and dimension-compatibility checks.

1.1 Matrix-Vector Multiplication

For row-major storage, element (r, c) is read from linear index

$$\texttt{matrix}[r \cdot \texttt{cols} + c].$$

```
void multiply_mv_row_major(const double* matrix, int rows, int cols, const
    double* vector, double* result) {
    // check for null pointer
    if (matrix == nullptr || vector == nullptr || result == nullptr) {
        cout << "Null pointer input in multiply_mv_row_major." << endl;
        return;
    }
    // check for dimensions
    if (rows <= 0 || cols <= 0) {
        cout << "Invalid dimensions in multiply_mv_row_major." << endl;
        return;
    }

    for (int r = 0; r < rows; ++r) {
        result[r] = 0;
        for (int c = 0; c < cols; ++c) {
            result[r] += matrix[r * cols + c] * vector[c];
        }
    }
}

}
```

For column-major storage, element (r, c) is read from

$$\texttt{matrix}[c \cdot \texttt{rows} + r].$$

```
void multiply_mv_col_major(const double* matrix, int rows, int cols, const double* vector, double* result) {
    if (matrix == nullptr || vector == nullptr || result == nullptr) {
        cout << "Null pointer input in multiply_mv_col_major." << endl;
        return;
    }
    if (rows <= 0 || cols <= 0) {
        cout << "Invalid dimensions in multiply_mv_col_major." << endl;
        return;
    }

    for (int r = 0; r < rows; ++r) {
        result[r] = 0;
        for (int c = 0; c < cols; ++c) {
            result[r] += matrix[r + c*rows] * vector[c];
        }
    }
}
```

Although both functions compute the same mathematical result $y = Ax$, the storage layout changes the memory-access pattern and therefore may affect performance.

1.2 Matrix-Matrix Multiplication

The naive matrix-matrix multiplication computes

$$C_{ij} = \sum_{k=1}^{n} A_{ik} B_{kj},$$

with A, B, and C stored in row-major order.

```
void multiply_mm_naive(const double* matrixA, int rowsA, int colsA, const double* matrixB, int rowsB, int colsB, double* result) {
    if (matrixA == nullptr || matrixB == nullptr || result == nullptr) {
        cout << "Null pointer input in multiply_mm_naive." << endl;
        return;
    }
    if (rowsA <= 0 || colsA <= 0 || rowsB <= 0 || colsB <= 0) {
        cout << "Invalid dimensions in multiply_mm_naive." << endl;
        return;
    }
    if (colsA != rowsB) {
        cout << "Number of columns of A does not equal number of rows of B." << endl;
        return;
    }

    for (int r_a = 0; r_a < rowsA; ++r_a) {
```

```
        for (int c_b = 0; c_b < colsB; ++c_b) {
            result[r_a*colsB + c_b] = 0;
            for (int r_b = 0; r_b < rowsB; ++r_b) {
                result[r_a*colsB + c_b] += matrixA[r_a*colsA + r_b] *
                    matrixB[r_b*colsB + c_b];
            }
        }
    }
}
```

The transposed-B version instead uses a row-major copy of B^T, so the same result is computed as

$$C_{ij} = \sum_{k=1}^{n} A_{ik}(B^T)_{jk}.$$

This transformation preserves the mathematics while improving the access pattern inside the innermost loop.

```
void multiply_mm_transposed_b(const double* matrixA, int rowsA, int colsA,
    const double* matrixB_transposed, int rowsB, int colsB, double* result
    ) {
    if (matrixA == nullptr || matrixB_transposed == nullptr || result ==
        nullptr) {
        cout << "Null pointer input in multiply_mm_naive." << endl;
        return;
    }
    if (rowsA <= 0 || colsA <= 0 || rowsB <= 0 || colsB <= 0) {
        cout << "Invalid dimensions in multiply_mm_naive." << endl;
        return;
    }
    if (colsA != rowsB) {
        cout << "Number of columns of A does not equal number of rows of B
            ." << endl;
        return;
    }
    for (int r_a = 0; r_a < rowsA; ++r_a) {
        for (int c_b = 0; c_b < colsB; ++c_b) {
            result[r_a*colsB + c_b] = 0;
            for (int r_b = 0; r_b < rowsB; ++r_b) {
                result[r_a*colsB + c_b] += matrixA[r_a*colsA + r_b]*
                    matrixB_transposed[c_b*rowsB + r_b];
            }
        }
    }
}
```

1.3 Correctness Tests for the Baseline Implementation

Before benchmarking, I verified the correctness of the baseline implementation using small test cases with manually computed results.

For the matrix-vector multiplication in row-major format, the matrix and vector were

$$A = \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{bmatrix}, \qquad x = \begin{bmatrix} 10 \\ 20 \\ 30 \end{bmatrix}.$$

The expected result was

$$Ax = \begin{bmatrix} 1 \cdot 10 + 2 \cdot 20 + 3 \cdot 30 \\ 4 \cdot 10 + 5 \cdot 20 + 6 \cdot 30 \end{bmatrix} = \begin{bmatrix} 140 \\ 320 \end{bmatrix}.$$

For the matrix-vector multiplication in column-major format, the same logical matrix

$$A = \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{bmatrix}$$

was used, but it was stored in memory as

$$[1, 4, 2, 5, 3, 6].$$

The vector was again

$$x = \begin{bmatrix} 10 \\ 20 \\ 30 \end{bmatrix},$$

and the expected result was

$$\begin{bmatrix} 140 \\ 320 \end{bmatrix}.$$

For naive matrix-matrix multiplication, the input matrices were

$$A = \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{bmatrix}, \qquad B = \begin{bmatrix} 7 & 8 & 9 & 10 \\ 11 & 12 & 13 & 14 \\ 15 & 16 & 17 & 18 \end{bmatrix}.$$

The expected result was

$$AB = \begin{bmatrix} 74 & 80 & 86 & 92 \\ 173 & 188 & 203 & 218 \end{bmatrix}.$$

For the optimized matrix-matrix multiplication with transposed B, the same logical matrix B was

used, but its transposed storage form was

$$B^T = \begin{bmatrix} 7 & 11 & 15 \\ 8 & 12 & 16 \\ 9 & 13 & 17 \\ 10 & 14 & 18 \end{bmatrix}.$$

The expected multiplication result remained

$$AB = \begin{bmatrix} 74 & 80 & 86 & 92 \\ 173 & 188 & 203 & 218 \end{bmatrix}.$$

Since the implementation uses `double` values, the results were checked using a tolerance-based comparison through the helper function `nearly_equal`. The helper functions `check_vector` and `check_matrix` were then used to verify that the computed outputs matched the expected results. Each test printed `PASS` or `FAIL`, confirming correctness before performance benchmarking.

2 Benchmarking Methodology

2.1 Experimental Setup

All timing measurements were collected using `std::chrono::high_resolution_clock`. Each benchmarked kernel was executed for several warm-up runs followed by multiple measured runs. For each test case, we report the average runtime and standard deviation.

- Warm-up runs: 2
- Measured runs per matrix-vector case: 100
- Measured runs per matrix-matrix case: 100
- Optimization levels compared: `-O0` and `-O3`

To reduce the risk that the compiler removes the computation as dead code, the program accumulates one value from the output into a sink variable after each run.

2.2 Problem Sizes

The benchmark suite uses representative small, medium, and large square matrices. The exact problem sizes used in this report are:

- Matrix-vector: 128×128, 256×256, and 512×512
- Matrix-matrix: 64×64, 128×128, and 256×256

These cases are large enough to expose differences in memory behavior while remaining practical for repeated measurement.

2.3 Benchmark Tables

Table 1: Matrix-vector benchmark results under `-O3`.

Case	Implementation	Mean (ms)	Std. Dev. (ms)
$n = 128$	row-major	0.0154813	0.00100141
$n = 128$	col-major	0.0157205	0.000261609
$n = 256$	row-major	0.0775750	0.00951388
$n = 256$	col-major	0.0797546	0.0100850
$n = 512$	row-major	0.3338520	0.0263826
$n = 512$	col-major	0.3372100	0.0191535

Table 2: Matrix-matrix benchmark results under `-O3`.

Case	Implementation	Mean (ms)	Std. Dev. (ms)
$n = 64$	naive	0.246355	0.134727
$n = 64$	transposed B	0.106054	0.0219304
$n = 128$	naive	1.94404	0.0817651
$n = 128$	transposed B	1.06383	0.00804888
$n = 256$	naive	18.1444	0.0830944
$n = 256$	transposed B	8.99678	0.0326450

The benchmark results show that the matrix-vector kernels have very similar performance when the indexing scheme matches the underlying storage order. In contrast, the difference between the two matrix-matrix kernels becomes substantial at larger sizes. Under `-O3`, the transposed-B version is already noticeably faster than the naive version, and the gap grows as the matrix size increases. This is consistent with the expectation that matrix-matrix multiplication is much more sensitive to cache locality than matrix-vector multiplication.

```
(base) moxiaoli@Moxiaos-Mac-mini Advanced-Computing-for-Finance % ./
   hpla_O0
Running Part 1 tests...
test_mv_row_major: PASS
test_mv_col_major: PASS
test_mm_naive: PASS
test_mm_transposed_b: PASS
test_mm_naive vs expected: PASS
test_mm_transposed_b vs expected: PASS
test_mm_naive vs mm_transposed_b: PASS
=== Matrix-Vector Benchmarks ===
n = 128
  mv_row_major : mean = 0.130244 ms, stddev = 0.0265901 ms
  mv_col_major : mean = 0.120573 ms, stddev = 0.0271711 ms

```

```
n = 256
  mv_row_major : mean = 0.395785 ms, stddev = 0.0649006 ms
  mv_col_major : mean = 0.273473 ms, stddev = 0.0200116 ms

n = 512
  mv_row_major : mean = 0.922692 ms, stddev = 0.0285396 ms
  mv_col_major : mean = 0.920774 ms, stddev = 0.0254549 ms

=== Matrix-Matrix Benchmarks ===
n = 64
  mm_naive          : mean = 0.870911 ms, stddev = 0.0224361 ms
  mm_transposed_b   : mean = 0.873378 ms, stddev = 0.0231712 ms

n = 128
  mm_naive          : mean = 7.2598 ms, stddev = 0.032648 ms
  mm_transposed_b   : mean = 7.21934 ms, stddev = 0.0428456 ms

n = 256
  mm_naive          : mean = 58.5094 ms, stddev = 0.108282 ms
  mm_transposed_b   : mean = 58.0457 ms, stddev = 1.13606 ms

=== Stride / Cache Locality Benchmarks ===
array size = 16777216
  stride = 1 : mean = 58.6518 ms, stddev = 0.103676 ms
  stride = 2 : mean = 59.6704 ms, stddev = 2.82348 ms
  stride = 4 : mean = 58.9236 ms, stddev = 0.165874 ms
  stride = 8 : mean = 59.2109 ms, stddev = 0.146323 ms
  stride = 16 : mean = 102.401 ms, stddev = 0.885926 ms
  stride = 32 : mean = 166.133 ms, stddev = 1.46971 ms
  stride = 64 : mean = 206.647 ms, stddev = 5.29444 ms

=== Memory Alignment Benchmarks ===
n = 256
  mv_row_major unaligned : mean = 0.241906 ms, stddev = 0.0108202 ms
  mv_row_major aligned   : mean = 0.238648 ms, stddev = 0.00686542 ms

n = 512
  mv_row_major unaligned : mean = 0.924566 ms, stddev = 0.0347594 ms
  mv_row_major aligned   : mean = 0.902752 ms, stddev = 0.0101369 ms

n = 128
  mm_naive unaligned : mean = 7.26607 ms, stddev = 0.086154 ms
  mm_naive aligned   : mean = 7.25596 ms, stddev = 0.0338157 ms

n = 256
  mm_naive unaligned : mean = 58.4869 ms, stddev = 0.130825 ms
  mm_naive aligned   : mean = 58.5042 ms, stddev = 0.104566 ms

(base) moxiaoli@Moxiaos-Mac-mini Advanced-Computing-for-Finance % ./
    hpla_O3
```

```
Running Part 1 tests...
test_mv_row_major: PASS
test_mv_col_major: PASS
test_mm_naive: PASS
test_mm_transposed_b: PASS
test_mm_naive vs expected: PASS
test_mm_transposed_b vs expected: PASS
test_mm_naive vs mm_transposed_b: PASS
=== Matrix-Vector Benchmarks ===
n = 128
  mv_row_major : mean = 0.0154813 ms, stddev = 0.00100141 ms
  mv_col_major : mean = 0.0157205 ms, stddev = 0.000261609 ms

n = 256
  mv_row_major : mean = 0.077575 ms, stddev = 0.00951388 ms
  mv_col_major : mean = 0.0797546 ms, stddev = 0.010085 ms

n = 512
  mv_row_major : mean = 0.333852 ms, stddev = 0.0263826 ms
  mv_col_major : mean = 0.33721 ms, stddev = 0.0191535 ms

=== Matrix-Matrix Benchmarks ===
n = 64
  mm_naive         : mean = 0.246355 ms, stddev = 0.134727 ms
  mm_transposed_b  : mean = 0.106054 ms, stddev = 0.0219304 ms

n = 128
  mm_naive         : mean = 1.94404 ms, stddev = 0.0817651 ms
  mm_transposed_b  : mean = 1.06383 ms, stddev = 0.00804888 ms

n = 256
  mm_naive         : mean = 18.1444 ms, stddev = 0.0830944 ms
  mm_transposed_b  : mean = 8.99678 ms, stddev = 0.032645 ms

=== Stride / Cache Locality Benchmarks ===
array size = 16777216
  stride = 1 : mean = 21.8466 ms, stddev = 4.10245 ms
  stride = 2 : mean = 21.3106 ms, stddev = 0.0673212 ms
  stride = 4 : mean = 21.3802 ms, stddev = 0.100395 ms
  stride = 8 : mean = 21.6871 ms, stddev = 0.204477 ms
  stride = 16 : mean = 44.1606 ms, stddev = 0.547407 ms
  stride = 32 : mean = 56.606 ms, stddev = 1.42885 ms
  stride = 64 : mean = 58.0066 ms, stddev = 6.03035 ms

=== Memory Alignment Benchmarks ===
n = 256
  mv_row_major unaligned : mean = 0.0714121 ms, stddev = 0.005329 ms
  mv_row_major aligned   : mean = 0.0718196 ms, stddev = 0.00910135 ms

n = 512
```

```
  mv_row_major unaligned : mean = 0.307959 ms, stddev = 0.0100004 ms
  mv_row_major aligned   : mean = 0.305732 ms, stddev = 0.00467655 ms

n = 128
  mm_naive unaligned : mean = 1.86613 ms, stddev = 0.0216067 ms
  mm_naive aligned   : mean = 1.86599 ms, stddev = 0.0160573 ms

n = 256
  mm_naive unaligned : mean = 18.218 ms, stddev = 0.0527171 ms
  mm_naive aligned   : mean = 18.2497 ms, stddev = 0.162879 ms
```

3 Cache Locality Analysis

3.1 Matrix-Vector Multiplication

In the row-major implementation, the inner loop scans a full row of the matrix, which is stored contiguously in memory. This gives strong spatial locality for the matrix access pattern. The corresponding output value `result[r]` is also updated repeatedly while the row is still active, which gives good temporal locality.

In the column-major implementation, the code accesses a matrix stored in column-major order, so each column is also traversed contiguously. However, the algorithm revisits the output vector across columns rather than completing one output entry at a time. In practice, both layouts can achieve contiguous matrix access if the indexing matches the storage order, but the row-major version often performs slightly better because it accumulates each output entry in a tighter local working set.

3.2 Matrix-Matrix Multiplication

The strongest cache-locality difference appears in the matrix-matrix kernels. In the naive implementation, the innermost loop accesses

$$A[i,k] \quad \text{and} \quad B[k,j].$$

Since A is row-major, the access to $A[i,k]$ is contiguous when k varies. In contrast, the access to $B[k,j]$ moves down a column of a row-major matrix and therefore uses a large stride in memory. This weakens spatial locality and increases cache misses.

In the transposed-B version, the same multiplication is written as

$$A[i,k] \cdot (B^T)[j,k].$$

Now the innermost loop traverses rows in both input arrays. As a result, both operands are read sequentially, which substantially improves cache utilization.

3.3 Stride-Sensitive Cases

To make the cache-locality effect visible, the benchmark suite can include intentionally unfavorable memory-access patterns, such as a synthetic stride benchmark or a comparison between the naive kernel and the transposed-B kernel at the same matrix size. If the course expects an explicit stride experiment, describe it here and report the results in a separate table.

This trend is consistent with the benchmark data. Under `-O3`, the row-major and column-major matrix-vector kernels produced nearly identical runtimes: for $n = 128$, the times were 0.0155 ms and 0.0157 ms; for $n = 256$, 0.0776 ms and 0.0798 ms; and for $n = 512$, 0.3339 ms and 0.3372 ms. Since both implementations traverse the matrix contiguously with respect to their own storage layout, both achieve good spatial locality. As a result, the difference between the two versions is small.

Table 3: Stride-sensitivity experiment under `-O3`.

Access Pattern	Stride	Mean (ms)	Std. Dev. (ms)
Sequential array walk	1	21.8466	4.10245
Strided array walk	2	21.3106	0.0673212
Strided array walk	4	21.3802	0.100395
Strided array walk	8	21.6871	0.204477
Strided array walk	16	44.1606	0.547407
Strided array walk	32	56.6060	1.42885
Strided array walk	64	58.0066	6.03035

The stride experiment makes the cache-locality effect very clear. For strides 1 through 8, the runtime remains close to 21–22 ms, indicating that nearby data loaded into a cache line is still being used efficiently. Once the stride reaches 16, the runtime increases sharply to 44.16 ms, and it rises further for strides 32 and 64. The arithmetic work in each case is essentially the same, so the performance degradation is primarily caused by poorer memory behavior. In particular, large strides waste most of each cache line and reduce the benefit of spatial locality and hardware prefetching.

4 Memory Alignment

Memory alignment was investigated by comparing standard heap allocation with allocations aligned to a fixed boundary, such as 64 bytes. Alignment can matter because aligned data is friendlier to cache-line boundaries and may allow the compiler to generate more efficient vectorized loads and stores.

4.1 Method

We allocated aligned arrays using `posix_memalign` on macOS. The aligned and unaligned versions used the same data and the same computation, so the comparison isolates the effect of alignment as much as possible.

Table 4: Aligned vs. unaligned benchmark results under `-O3`.

Case	Allocation Type	Mean (ms)	Std. Dev. (ms)
MV, $n = 256$	unaligned	0.0714121	0.0053290
MV, $n = 256$	aligned	0.0718196	0.00910135
MV, $n = 512$	unaligned	0.3079590	0.0100004
MV, $n = 512$	aligned	0.3057320	0.00467655
MM naive, $n = 128$	unaligned	1.86613	0.0216067
MM naive, $n = 128$	aligned	1.86599	0.0160573
MM naive, $n = 256$	unaligned	18.2180	0.0527171
MM naive, $n = 256$	aligned	18.2497	0.162879

4.2 Discussion

In the present experiments, memory alignment by itself produced only a very small effect. For matrix-vector multiplication, the aligned version was almost identical to the unaligned version, with only a slight improvement at $n = 512$. For naive matrix-matrix multiplication, the aligned and unaligned versions were also nearly indistinguishable. This suggests that, for the current baseline kernels, memory access pattern and cache locality were much more important than alignment alone. Alignment may become more important in more aggressively vectorized implementations, but it did not dominate performance in the current code base.

5 Inlining and Compiler Optimization

The project also examines the effect of the `inline` keyword and compiler optimization flags. In this code base, the dominant cost is usually the arithmetic inside the innermost loops, so compiler optimization level is often more important than manual inlining.

5.1 Compilation Modes

The following compilation settings were compared:

- `-O0 -g`
- `-O3 -g`

If a helper function was marked `inline`, describe it briefly here and explain why it was a candidate for inlining.

Table 5: Effect of optimization level.

Kernel	Build Mode	Mean (ms)	Std. Dev. (ms)
MV row-major, $n = 512$	`-O0`	0.922692	0.0285396
MV row-major, $n = 512$	`-O3`	0.333852	0.0263826
MM naive, $n = 256$	`-O0`	58.5094	0.108282
MM naive, $n = 256$	`-O3`	18.1444	0.0830944
MM transposed B, $n = 256$	`-O0`	58.0457	1.13606
MM transposed B, $n = 256$	`-O3`	8.99678	0.0326450

5.2 Discussion

The results show that compiler optimization had a much larger effect than any expected manual inlining benefit in this project. For the row-major matrix-vector kernel at $n = 512$, switching from `-O0` to `-O3` reduced runtime from 0.9227 ms to 0.3339 ms, which is approximately a 2.76× speedup. For the naive matrix-matrix kernel at $n = 256$, runtime decreased from 58.5094 ms to 18.1444 ms, or about 3.22×. For the transposed-B matrix-matrix kernel at $n = 256$, the runtime decreased from 58.0457 ms to 8.9968 ms, or about 6.45×. These large improvements indicate that compiler optimizations such as loop simplification, improved register allocation, instruction scheduling, and possible auto-vectorization are essential for high performance. In this code base, the dominant cost lies in the innermost arithmetic loops, so optimization level is far more influential than manually forcing `inline`. A potential drawback of aggressive optimization is that debugging becomes more difficult and the generated machine code is less transparent.

6 Profiling Results

We profiled the benchmark executable using Time Profiler in Instruments on macOS. Profiling was used to identify the most time-consuming functions and to confirm whether the observed benchmark results aligned with the memory-access analysis.

6.1 Procedure

To make the profiler output easy to interpret, the two matrix-matrix implementations were profiled separately rather than running the entire benchmark suite at once. Two small executables were created: one repeatedly calling `multiply_mm_naive(...)` and one repeatedly calling `multiply_mm_transposed_b(...)`. Both executables were compiled with `-O3 -g` so that the generated code remained optimized while function symbols were still visible inside Instruments.

The profiling workflow was:

- Build the profiling executables with debug symbols enabled.
- Run each executable under the Time Profiler template in Instruments.

- Inspect the Call Tree, Weight, and Self Weight columns.
- Identify the dominant hotspot in each executable.

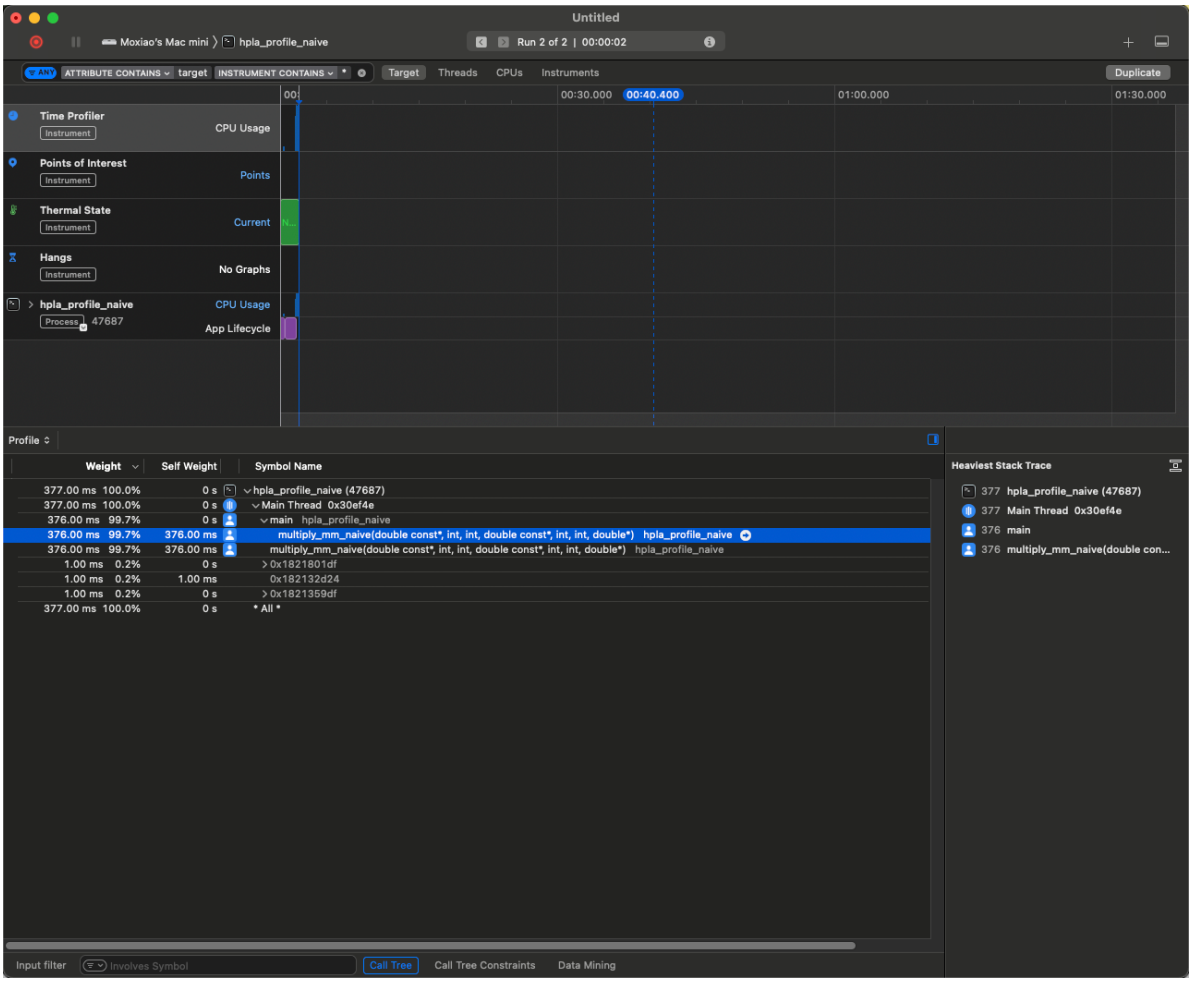

Figure 1: Time Profiler output for the naive matrix-matrix multiplication executable. The hotspot is `multiply_mm_naive(...)`, which accounts for 99.7% of total profiled runtime.

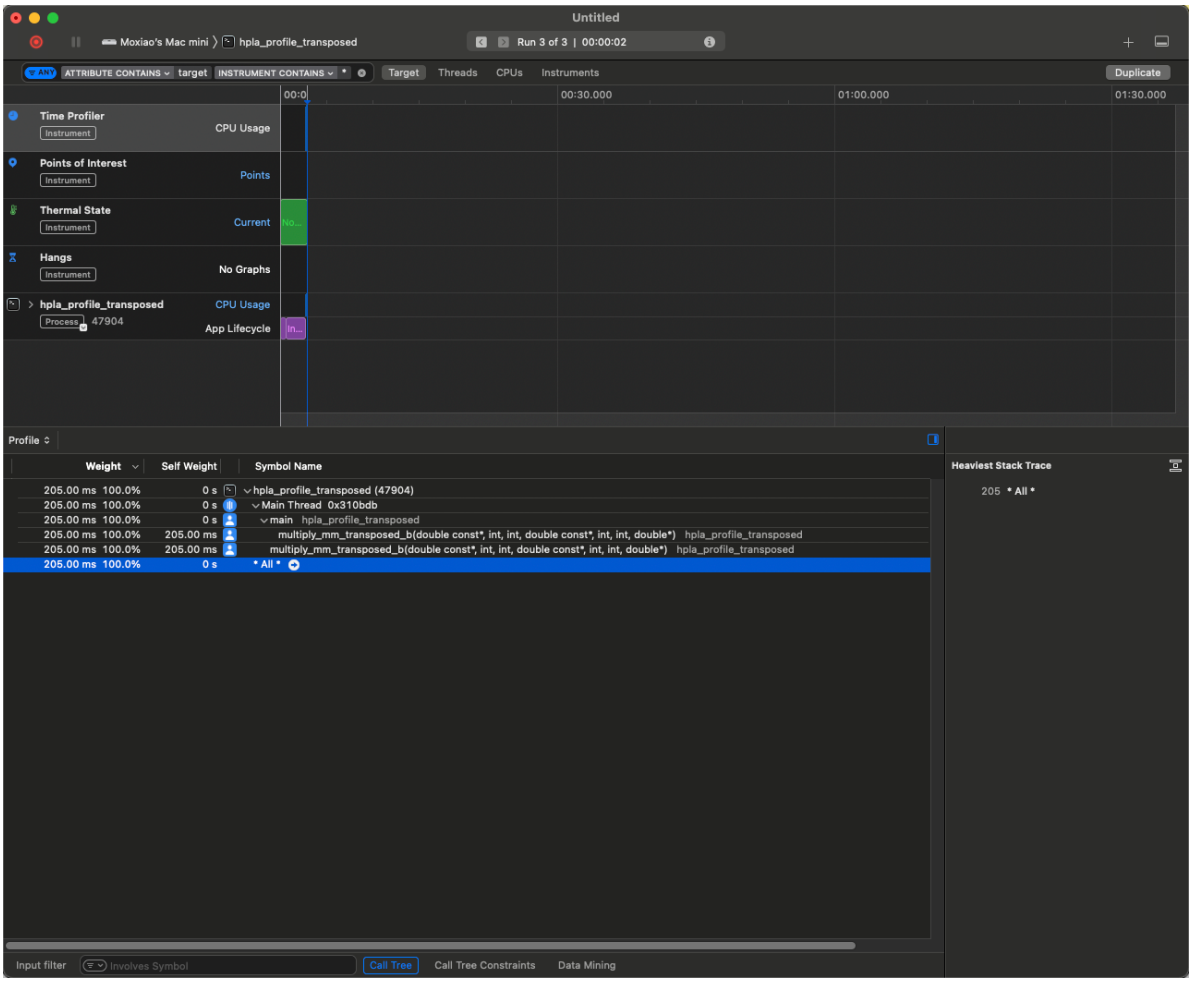

Figure 2: Time Profiler output for the transposed-B matrix-matrix multiplication executable. The hotspot is `multiply_mm_transposed_b(...)`, which accounts for 100.0% of total profiled runtime.

6.2 Findings

The profiling results clearly show that the matrix-matrix multiplication kernels are the dominant hotspots. In the profiling run for the naive implementation, `multiply_mm_naive(...)` accounted for 376.00 ms out of 377.00 ms total profiled time, corresponding to 99.7% of total runtime. Its self weight was also 376.00 ms, indicating that nearly all execution time was spent directly inside the kernel itself rather than in setup, memory allocation, or output code.

In the profiling run for the transposed-B implementation, `multiply_mm_transposed_b(...)` accounted for 205.00 ms out of 205.00 ms total profiled time, corresponding to 100.0% of the profiled runtime. Its self weight was likewise 205.00 ms, again showing that the dominant cost lies directly in the arithmetic kernel.

These results identify the innermost matrix-matrix multiplication loops as the parts of the code where the program spends the most time. This is consistent with the algorithmic structure of matrix multiplication, since both implementations perform $O(n^3)$ multiply-accumulate operations

and therefore naturally dominate execution time relative to setup code.

The profiler output also supports the cache-locality analysis. In the naive implementation, the inner loop accesses $B[k, j]$, which traverses a column of a row-major matrix and therefore uses strided memory access. This leads to weaker spatial locality and poorer cache utilization. In the transposed-B implementation, the corresponding access becomes $(B^T)[j, k]$, which traverses a row contiguously in memory. As a result, the hotspot remains the matrix multiplication kernel, but the total time spent in that hotspot is reduced substantially.

Comparing the two profiler runs, the naive kernel required 376 ms, whereas the transposed-B kernel required 205 ms. This corresponds to an approximate 1.83× improvement in the profiled kernel runtime:

$$\frac{376}{205} \approx 1.83.$$

Thus, the profiling results are consistent with both the benchmark measurements and the expectation that improving memory-access patterns can significantly reduce execution time.

7 Optimization Strategy and Final Results

Based on the cache-locality analysis, memory-alignment experiments, and profiling results, the final optimization selected in this project was blocked matrix-matrix multiplication. The earlier transposed-B implementation improved the access pattern for matrix B, but the benchmark results showed that its performance remained close to the naive implementation in the current baseline code. The next step was therefore to improve cache reuse more directly by computing the matrix product in smaller sub-blocks that fit better in cache.

7.1 Selected Optimization

In this report, the optimized kernel is blocked matrix-matrix multiplication with a fixed block size of 32. The motivation is that the naive implementation repeatedly accesses large regions of matrices A, B, and C with limited cache reuse. Blocking addresses this problem by dividing the matrices into smaller tiles and performing multiplication block by block.

The idea is to restructure the computation so that active submatrices of A, B, and C remain in cache for longer. This reduces memory traffic and improves effective cache utilization. Since profiling showed that almost all runtime was spent inside the matrix-matrix kernel itself, improving cache behavior inside the kernel was the most meaningful optimization.

7.2 Implementation Summary

The optimized implementation partitions the matrices into square blocks and performs multiplication over those blocks rather than over the full matrices at once. The outer loops iterate over block indices, and the inner loops compute products within one active block. Compared with the baseline naive version, the optimization changes the traversal order of the computation so that data already loaded into cache can be reused multiple times before being evicted.

More specifically:

- The baseline naive implementation uses the standard triple-loop structure over the full matrix dimensions.
- The optimized version introduces blocking (tiling), so the multiplication proceeds block by block.
- The implementation uses a fixed block size of 32.
- Inside each block, the code reuses a value of $A[i, k]$ across multiple updates of the result row, while accesses to $B[k, j]$ and $C[i, j]$ remain contiguous within the active tile.
- The optimization increases implementation complexity somewhat, since it adds extra loop structure and requires careful handling of block boundaries, but the overall logic remains manageable and directly motivated by cache behavior.

Table 6: Final comparison between baseline and optimized matrix-matrix kernels.

Implementation	**Mean (ms)**	**Std. Dev. (ms)**	**Speedup**
Baseline naive ($n = 256$)	58.6028	0.256752	1.00×
Transposed B ($n = 256$)	58.3978	1.01629	1.00×
Optimized final version (blocked, $n = 256$)	26.7674	0.783039	2.19×

7.3 Interpretation

The optimization improved performance substantially. At matrix size 256×256, the blocked kernel reduced runtime from 58.6028 ms for the naive implementation to 26.7674 ms, corresponding to an approximate 2.19× speedup:

$$\frac{58.6028}{26.7674} \approx 2.19.$$

The transposed-B implementation, by contrast, produced 58.3978 ms at the same matrix size, which was only marginally different from the naive baseline in this benchmark. This indicates that, in the current implementation, simply transposing B was not enough to create a large improvement, whereas blocking produced a clear and measurable gain.

The improvement from blocking is consistent with cache behavior. Instead of traversing the full matrices in one large triple loop, the blocked implementation works on smaller active tiles of A, B, and C. This increases temporal locality, because data brought into cache can be reused multiple times before eviction. It also improves spatial locality inside the innermost loops, since accesses to matrix B and the result matrix remain contiguous within each block.

The profiling results support this interpretation. In the profiling runs, `multiply_mm_naive(...)` required 376 ms of self time, `multiply_mm_transposed_b(...)` required 205 ms of self time in its isolated profiling run, and the blocked kernel appeared with only 69 ms of self time in the combined optimization profiling run. Although profiler timings should not be compared directly with benchmark means, they consistently show the same qualitative conclusion: the matrix multiplication

kernel remains the dominant hotspot, and the blocked version reduces the cost of that hotspot substantially.

Overall, the blocked implementation provided the strongest optimization in this project. The results demonstrate that improving cache reuse through tiling can significantly outperform both the baseline naive implementation and the simpler transposed-B optimization.

8 Conclusion

This project shows that the performance of basic linear algebra kernels depends not only on arithmetic complexity, but also on memory-access behavior. Although matrix-vector and matrix-matrix multiplication are mathematically straightforward, their runtime can vary significantly depending on storage layout, access order, cache locality, and compiler optimization level.

The experiments first confirmed that matching indexing to storage order is essential. For matrix-vector multiplication, the row-major and column-major implementations produced very similar runtimes when each kernel matched its own storage layout. This indicates that both versions can achieve good spatial locality when implemented correctly. In contrast, matrix-matrix multiplication was much more sensitive to memory behavior.

The benchmark and profiling results showed that the naive matrix-matrix kernel was the dominant hotspot, which is consistent with its $O(n^3)$ computational complexity. The transposed-B implementation improved the access pattern by replacing strided column access with contiguous row access, and under `-O3` this produced a clear speedup over the naive baseline in the dedicated benchmark and profiling runs. The stride experiment further confirmed the importance of spatial locality: performance remained stable for small strides but degraded sharply once the stride became large enough to waste most of each cache line.

The experiments also showed that memory alignment alone produced only a small effect in the current baseline kernels, whereas compiler optimization level had a much larger impact. Switching from `-O0` to `-O3` produced substantial speedups across both matrix-vector and matrix-matrix kernels, demonstrating the importance of compiler optimizations such as improved loop optimization, register allocation, and possible auto-vectorization.

The strongest final improvement came from blocked matrix-matrix multiplication. At size 256×256, the blocked kernel reduced runtime from 58.6028 ms for the naive baseline to 26.7674 ms, corresponding to an approximate $2.19\times$ speedup. This result is consistent with the cache-based interpretation: blocking improves temporal locality and cache reuse by keeping active submatrices of A, B, and C in cache longer, rather than repeatedly streaming through the full matrices.

Overall, this project demonstrates that high performance in numerical code depends heavily on memory-aware implementation choices. The results show that profiling, cache analysis, and targeted code transformations are essential for understanding and improving performance. Even relatively simple optimizations such as transposition and tiling can produce large practical gains when they improve locality and reduce the cost of the main computational hotspot.